LeetCode HOT 100

PART 2

Two Pointers

双指针

4. 移动零 (No.283) EASY

给定一个数组 `nums`，编写一个函数将所有 `0` 移动到数组的末尾，同时保持非零元素的相对顺序。

请注意 ，必须在不复制数组的情况下原地对数组进行操作。

示例 1:

> **输入：** `nums = [0,1,0,3,12]`
> **输出：** `[1,3,12,0,0]`

示例 2:

> **输入：** `nums = [0]`
> **输出：** `[0]`

提示:

- `1 <= nums.length <= 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

进阶： 你能尽量减少完成的操作次数吗?

4. 移动零 (No.283) EASY

思路

快指针负责往前找到下一个非0数，慢指针的位置是下一个非0数在新数组中的位置（要么和快指针重合，要么停留在0）。快指针每找到一个非0数，就与慢指针交换元素。一趟遍历下来所有非0数都在新数组的正确位置，所有0都被换到尾部。

```
class Solution(object):
    def moveZeroes(self, nums):
        """
        :type nums: List[int]
        :rtype: None Do not return anything, modify nums in-place instead.
        """
        slow, fast, n = 0, 0, len(nums)
        while fast < n:
            if nums[fast]:  # 遇到非0数
                nums[fast], nums[slow] = nums[slow], nums[fast]
                slow += 1
            fast += 1
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(1)$

5. 盛水最多的容器 (No.11) MEDIUM

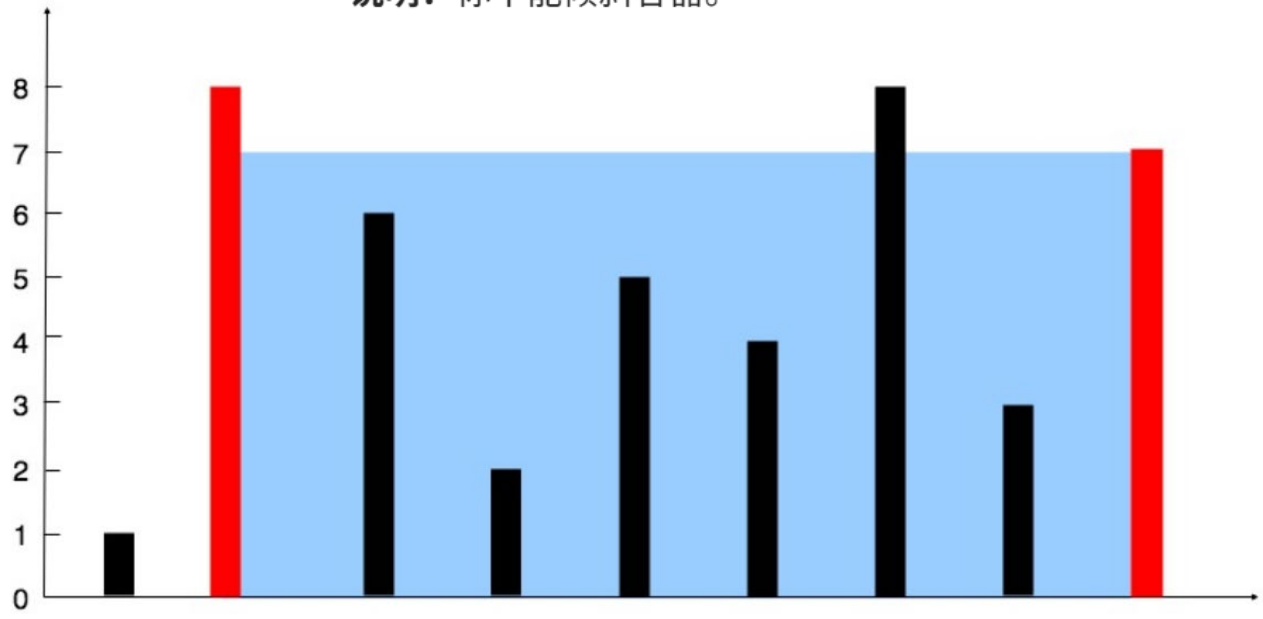
给定一个长度为 `n` 的整数数组 `height` 。有 `n` 条垂线，第 `i` 条线的两个端点是 `(i, 0)` 和 `(i, height[i])` 。

找出其中的两条线，使得它们与 `x` 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。

说明： 你不能倾斜容器。

示例 1：



> **输入：** [1,8,6,2,5,4,8,3,7]
> **输出：** 49
> **解释：** 图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下，容器能够容纳水（表示为蓝色部分）的最大值为 49。

示例 2：

> **输入：** height = [1,1]
> **输出：** 1

提示：

- `n == height.length`
- `2 <= n <= 10^5`
- `0 <= height[i] <= 10^4`

5. 盛水最多的容器 (No.11) MEDIUM

思路

左指针和右指针逐渐向中间靠拢，每次更新最大容量，然后移动高度较小的指针向内移动。

方法正确性的原因：木桶效应，移动高度较大的指针后的容量一定比移动前更小（高度不增，宽度减小）。

```
class Solution(object):
    def maxArea(self, height):
        """
        :type height: List[int]
        :rtype: int
        """
        left, right = 0, len(height)-1
        res = 0
        while left < right:
            res = max(res, (right - left) * min(height[left], height[right]))
            if height[left] < height[right]:
                left += 1
            else:
                right -= 1
        return res
```

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(1)$

6. 三数之和 (No.15) MEDIUM

给你一个整数数组 `nums` ，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k` ，同时还满足 `nums[i] + nums[j] + nums[k] == 0` 。请你返回所有和为 `0` 且不重复的三元组。

注意： 答案中不可以包含重复的三元组。

示例 1：

> **输入：** nums = [-1,0,1,2,-1,-4]
> **输出：** [[-1,-1,2],[-1,0,1]]
> **解释：**
> nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0 。
> nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0 。
> nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0 。
> 不同的三元组是 [-1,0,1] 和 [-1,-1,2] 。
> 注意，输出的顺序和三元组的顺序并不重要。

示例 2：

> **输入：** nums = [0,1,1]
> **输出：** []
> **解释：** 唯一可能的三元组和不为 0 。

示例 3：

> **输入：** nums = [0,0,0]
> **输出：** [[0,0,0]]
> **解释：** 唯一可能的三元组和为 0 。

提示：

- `3 <= nums.length <= 3000`
- `-10^5 <= nums[i] <= 10^5`

6. 三数之和 (No.15) MEDIUM

思路

官方解答的写法看着有点乱，个人觉得这个思路更清晰一些。

先数组排序。

三个指针p1,p2,p3，先固定p1，然后p2和p3在p1右侧使用左右指针向内移动，三数之和小于0就左指针右移，大于0就右指针左移。

为了防止枚举重复，指针移动时要确保不和上一次枚举的数相同。

时间复杂度：$\mathcal{O}(n^2)$

空间复杂度：$\mathcal{O}(n)$

```python
class Solution(object):
    def threeSum(self, nums):
        """
        :type nums: List[int]
        :rtype: List[List[int]]
        """
        res = []
        p1, n = 0, len(nums)
        sorted_nums = sorted(nums)
        while p1 < n:
            p2, p3 = p1 + 1, n - 1
            while p2 < p3:
                if sorted_nums[p1] + sorted_nums[p2] + sorted_nums[p3] == 0:
                    res.append([sorted_nums[p1], sorted_nums[p2], sorted_nums[p3]])
                    p2 += 1
                    p3 -= 1
                    # 为了防止重复，下一次枚举的数必须跟上次不一样
                    while p2 < p3 and sorted_nums[p2] == sorted_nums[p2-1]:
                        p2 += 1
                    while p2 < p3 and sorted_nums[p3] == sorted_nums[p3+1]:
                        p3 -= 1
                elif sorted_nums[p1] + sorted_nums[p2] + sorted_nums[p3] > 0:
                    p3 -= 1
                else:
                    p2 += 1
            p1 += 1
            # 为了防止重复，下一次枚举的数必须跟上次不一样
            while p1 < n and sorted_nums[p1] == sorted_nums[p1-1]:
                p1 += 1
        return res
```

7. 接雨水 (No.42) HARD

给定 `n` 个非负整数表示每个宽度为 `1` 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1：

> **输入：** height = [0,1,0,2,1,0,1,3,2,1,2,1]
> **输出：** 6
> **解释：** 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

示例 2：

> **输入：** height = [4,2,0,3,2,5]
> **输出：** 9

提示：

- `n == height.length`
- `1 <= n <= 2 * 10^4`
- `0 <= height[i] <= 10^5`

7. 接雨水 (No.42) HARD

Solution 1 – 双指针

思路

维护leftmax和rightmax记录左右指针目前遇到最高的柱子高度，然后左右指针向内移动，每次移动计算新增的水量。移动left时，必有leftmax<rightmax，因此leftmax就是新增水量的上界面，leftmax-height[left]就是新增水量，right同理。这一命题可以被证明。

> **命题：左右指针 left 和 right 满足** `height[left] < height[right]` **时有** `leftmax < rightmax`**。同理，** `height[left] >= height[right]` **时有** `leftmax >= rightmax`
>
> 若 `leftmax = height[left]`，那么 `leftmax = height[left] < height[right] < rightmax` 得证。
>
> 若 `leftmax > height[left]`，那么在 `left` 走到当前位置之前，存在 `left0` 满足 `leftmax = height[left0]`。由于 `left` 从 `left0` 曾经移开过（否则无法到达当前位置），说明在右侧存在 `right0` 使得 `height[left0] < height[right0]`，则 `leftmax = height[left0] < height[right0] <= rightmax`，得证。

```python
class Solution(object):
    def trap(self, height):
        """
        :type height: List[int]
        :rtype: int
        """
        n = len(height)
        left, right = 0, n - 1
        leftmax, rightmax = height[left], height[right]
        res = 0
        while left < right:
            if height[left] < height[right]:
                left += 1
                leftmax = max(leftmax, height[left])
                res += leftmax - height[left]
            else:
                right -= 1
                rightmax = max(rightmax, height[right])
                res += rightmax - height[right]
        return res
```

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$

7. 接雨水 (No.42) HARD

Solution 2 – 最高点分割

思路

Solution 1中的命题比较难想到，如果没办法确定两侧柱子较矮的一端，更粗暴的办法是找到最高的柱子，然后从中间往两端遍历，这样两端的柱子必然是较矮的一端，从而是新增水量的上界面。以左端为例，就可以直接计算水量为：leftmax-height[left]。

```
class Solution(object):
    def trap(self, height):
        """
        :type height: List[int]
        :rtype: int
        “”“
        res = 0
        leftmax, rightmax = 0, 0
        max_index = height.index(max(height))
        n = len(height)
        for left in range(max_index):  #最高处左侧从左往右遍历
            leftmax = max(leftmax, height[left])
            res += leftmax - height[left]
        for right in range(n - 1, max_index, -1):  #最高处右侧从右往左遍历
            rightmax = max(rightmax, height[right])
            res += rightmax - height[right]
        return res
```

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$

7. 接雨水 (No.42) HARD

Solution 3 – 两趟遍历

思路

同Solution 1&2的思路，其实只要找到每次新增水量的上界面就能计算结果了。因此从左向右遍历一次，从右向左遍历一次，分别记录每个位置的历史最大值leftmax和rightmax，对于位置i，min(leftmax[i], rightmax[i])就是上界面。

```
class Solution(object):
    def trap(self, height):
        """
        :type height: List[int]
        :rtype: int
        """
        res = 0
        n = len(height)
        left, right = 0, 0
        leftmax, rightmax = [], []
        for i in range(n):  #从左到右遍历记录leftmax数组
            left = max(left, height[i])
            leftmax.append(left)
        for i in range(n-1, -1, -1):  #从右到左遍历记录rightmax数组
            right = max(right, height[i])
            rightmax.append(right)
        rightmax = list(reversed(rightmax))
        for i in range(n):  #计算res
            res += min(leftmax[i], rightmax[i]) - height[i]
        return res
```

时间复杂度：$O(n)$

空间复杂度：$O(n)$

7. 接雨水 (No.42) HARD

Solution 4 – 单调栈

思路

有凹槽才能装水，遍历每个位置将其视为凹槽底部，希望找到右侧第一个比凹槽高的位置。

单调栈：栈中元素为下标，对应的高度递减，其中栈底就是前面的leftmax。每当遇到比栈顶元素高的柱子，凹槽就出现了：新柱子作为右壁，弹出栈顶元素做凹槽，新栈顶是左壁，凹槽底宽是左右壁距离，高度是左右壁较小值-凹槽高度。Solution 1-3的水量都是一列一列算的，这里水量是从凹槽底部开始“一凹一凹”算的。

```python
class Solution(object):
    def trap(self, height):
        """
        :type height: List[int]
        :rtype: int
        """
        res = 0
        n = len(height)
        stack = []  # 单调栈，栈内元素为下标，对应高度递减
        for i, h in enumerate(height):
            # 如果当前高度比栈顶元素高，那么栈顶位置形成凹槽
            while stack and h > height[stack[-1]]:
                top = stack.pop()  # 栈顶元素作为凹槽
                if not stack:
                    break  # 栈底是leftmax，如果弹出说明没有左壁，不能形成凹槽
                left = stack[-1]  # 此时新栈顶是左壁
                # 新右壁出现时底宽是i-left-1
                res += (i - left - 1) * (min(height[left], h) - height[top])
            stack.append(i)
        return res
```

TIPS：当需要找接下去第一个满足某条件的位置 → 单调栈

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(n)$